

Deep learning model for precise and rapid prediction of tomato maturity based on image recognition

Muhammad Waseem^a, Muhammad Muzzammil Sajjad^b, Laraib Haider Naqvi^c,
Yaqoob Majeed^{d,*}, Tanzeel Ur Rehman^a, Tayyaba Nadeem^e

^a Department of Biosystems Engineering, Auburn University, AL 36890, USA

^b Department of Food Engineering, University of Agriculture, Faisalabad, Pakistan

^c Department of Mechanical Engineering, University of Georgia, Athens, USA

^d Department of Electrical Engineering and Computer Science, University of Wyoming, USA

^e Department of Food Technology, National Institute of Food Science and Technology, University of Agriculture, Faisalabad, Pakistan

ARTICLE INFO

Keywords:

Efficient net

Resnet

Maturity indices

Tomatoes

Pruning

Quantization

ABSTRACT

Tomato maturity plays a pivotal role in optimizing harvest timing and ensuring product quality, but current methods struggle to achieve high accuracy along computational efficiency simultaneously. Existing deep learning approaches, while accurate, are often too computationally demanding for practical use in resource-constrained agricultural settings. In contrast, simpler techniques fail to capture the nuanced features needed for precise classification. This study aims to develop a computationally efficient tomato classification model using the ResNet-18 architecture optimized through transfer learning, pruning, and quantization techniques. Our objective is to address the dual challenge of maintaining high accuracy while enabling real-time performance on low-power edge devices. Then, these models were deployed on an edge device to investigate their performance for tomato maturity classification. The quantized model achieved an accuracy of 97.81 %, offering superior efficiency with an average classification time of 0.000975 s per image. The pruned and auto-tuned model also demonstrated significant improvements in deployment metrics, further highlighting the benefits of optimization techniques. These results underscore the potential for a balanced solution that meets the accuracy and efficiency demands of modern agricultural production, paving the way for practical, real-world deployment in resource-limited environments.

1. Introduction

Tomatoes (*Solanum lycopersicum*), a linchpin in global agricultural production, necessitate a comprehensive understanding and application of maturity indices. These indices ranging from Green, Breaker, Pink, Light Red, Red, and Over mature are essential for optimal cultivation and distribution [10,4]. These indices, pivotal in determining harvest timing, significantly impact the produce's quality, nutritional value, and economic returns [30]. The stage of maturity influences not only the organoleptic properties of

* Corresponding author.

E-mail addresses: m.w@auburn.edu (M. Waseem), muzzammil.sajjad@outlook.com (M.M. Sajjad), lh77939@uga.edu (L.H. Naqvi), ymajeed@uwyo.edu (Y. Majeed), tur0001@auburn.edu (T.U. Rehman), tayyabasofficial@gmail.com (T. Nadeem).

<https://doi.org/10.1016/j.foodp.2025.100060>

Received 16 April 2025; Received in revised form 2 June 2025; Accepted 10 July 2025

Available online 12 July 2025

2950-0699/© 2025 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

tomatoes such as flavor and texture, but also their processing suitability and shelf life [2,29]. Accurate maturity assessment minimizes wastage, optimizes logistics, and ensures a consistent supply of high-quality tomatoes, a critical factor in addressing the nutritional needs of the world's growing population [27,5].

The integration of automated classification systems such as AI-driven maturity indices based on image processing and machine learning-based quality assessments in agriculture signifies a revolutionary shift in crop management and quality control strategies [34]. These AI-driven approaches provide scalable and objective alternatives to manual assessments, enhancing consistency and reducing human error. Moreover, while deep learning techniques have improved classification accuracy, many such models suffer from high computational demands that limit their scalability in resource-constrained environments. Filip et al., [6,9]. These technologies, provide the potential to make significant advancements in the postharvest handling of fruits and vegetables [18]. Incorporating such systems empowers farmers and distributors with data-driven insights, fostering increased productivity and sustainable agricultural practices.

Mputu et al. [20] introduced ML algorithms for tomato quality sorting, achieving a classification accuracy of 89 % and it does not fully address computational efficiency issues for real-time applications. Qasrawi et al., [24] achieved an accuracy of 70.3 % with neural networks and 68.9 % with logistic regression models for tomato classification, particularly mature-stage tomatoes with diseases, which demands considerable computational resources, rendering it less practical for deployment in resource-constrained environments. Vo et al. [33] applied YOLOv9 for automating tomato ripeness classification (Unripe, Semi Ripe, Fully Ripe), achieving a precision of 0.856, recall of 0.832, and mAP50 of 0.882. Nath and Roy [21] reached 80 % accuracy using CNN for sentiment analysis to classify text into positive, negative, or neutral sentiments of rotten tomatoes. Zhang et al. [36] developed a YOLOv5-based algorithm for tomato detection and pose estimation achieving 82.4 % accuracy on unoccluded tomatoes for harvesting, the complexity of YOLO increases computational overhead, which hinders its scalability. Quach et al. [25] used MobileNet models for Unripe, ripe, Old, and damaged tomato classification and visual explanations, the study does not thoroughly address potential trade-offs between model efficiency and accuracy, to its practical scalability. Chen et al. [1] worked on the tomatoes classification system for green, breaker, turning, pink, light red, and red stages, achieving an accuracy of 81.3 % from Denesenet, 83.5 % from ResNet, 69.1 % from MobileNet, 67.4 % from AlexNet, 58.3 % from Shufflenet, 85.6 % from SqueezeNet based on open source dataset. The reliance on standard architectures and an open-source dataset suggests that the method not be fully optimized. Pacheco and López [22] worked on tomato classification for green, breaker, turning, pink, light red, and red stages using K-NN and MLP neural networks based on color, achieving an accuracy of 90 %. The heavy reliance on color-based features restrict the method's robustness under different environmental conditions. de Oliveira et al. [3] achieved 92 % accuracy in classifying organic versus non-organic tomatoes using Decision Trees on mass spectrometry data, its reliance on expensive and time-consuming mass spectrometry limits practical scalability and real-world applicability. Despite the high accuracy, the approach is less practical for large-scale or real-time applications due to its dependence on expensive and time-consuming mass spectrometry. Garcia et al. [7] automated ripeness classification of tomatoes, the study's reliance on SVM yielded 83.39 % accuracy on a limited dataset of 900 images, restricting its effectiveness and scalability in real-world applications. The above-mentioned studies faced limitations related to scalability, accuracy, and practicality. Few of the studies relied on limited open-source datasets, making them less effective in real-world applications. Machine learning models such as SVM and logistic regression, which use hand-crafted features, further restricted the generalizability of these approaches. Additionally, while deep learning-based methods offer greater accuracy, they are computationally intensive, demanding substantial processing power, which limits their practical adoption in resource-constrained environments. Therefore, it is essential to develop models that balance classification accuracy with computational and energy efficiency for real-world deployment. In recent years, techniques such as pruning and quantization have been introduced to enhance the computational efficiency of deep learning models. Pruning helps to reduce the size of a neural network by removing less significant weights, allowing the model to maintain accuracy while using fewer resources. On the other hand, quantization reduces the precision of the model's parameters, such as converting 32-bit floating-point numbers to 8-bit integers, which lowers the computational and memory requirements without significantly sacrificing performance. These approaches are especially beneficial for deploying models in resource-constrained environments.

Therefore, the primary goal of this research is to develop a computationally efficient tomato classification model using the ResNet-18 architecture. This study seeks to overcome the challenges posed by computationally intensive deep learning networks using pruning and quantization approaches, enabling their deployment in resource-constrained agricultural environments. The first objective of this research is to optimize ResNet-based model to accurately classify tomatoes based on maturity stages. The second objective is to make the ResNet model lighter and easier to deploy on edge devices by leveraging pruning and quantization approaches. The final objective is to deploy the optimized model on the edge device to investigate its performance capabilities for real-time classification of tomato maturity. The results of this study demonstrate the real-time utility of the deep learning model, showcasing its ability to operate efficiently on a low-power edge device.

Our work presents an integrated end-to-end system that not only captures images with a highly controlled and reproducible imaging setup but also tailors a standard ResNet-18 architecture through innovative optimization techniques for deployment on resource-constrained devices. Our contribution lies in the novel integration of these components with advanced network optimization specifically, a combined pipeline of magnitude-based unstructured pruning and quantization to substantially reduce computational demands (achieving a 66.92 % parameter reduction) without compromising classification accuracy. The system's design to capture multiple image orientations under varying lighting conditions further enhances the robustness of the tomato maturity classification, thereby offering a unique and practical solution tailored for edge device deployment.

Illustration of Research Methodology for this study

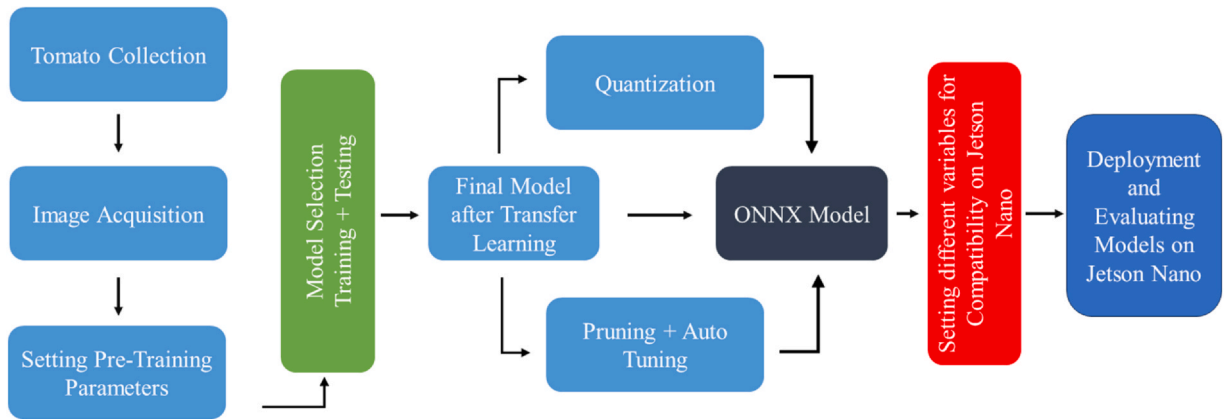


Fig. 1. Illustration of research methodology for this study.

2. Material & methodology

The methodology employed in this work, as illustrated in Fig. 1, was comprised of data collection using Raspberry color camera V2.1, deep learning model training along with optimization and quantization, and model deployment and testing on the Jetson platform.

2.1. Image acquisition

To collect the image dataset for this study, *Solanum lycopersicum* tomato (Rio Grande) variety was procured from the local market. A total of five vendors were randomly selected to collect the tomato samples to ensure the variability of the dataset. A total of 25 tomato samples were chosen for each maturity stage (Green, Breaker, Pink, Light Red, Red, and Over mature) as per USDA standards [32] with consideration provided by Serge Ndayitabi [28] and [7] making up to a total of 150 tomato samples, as given in Table 1. The selection criteria for the tomatoes were based on USDA standards: Green (completely green), Breaker (10 % or less color break), Pink (30 %-60 % pink/red), Light Red (60 %-90 % red), Red (over 90 % red), and Over Mature (signs of over-ripeness). To ensure accurate classification, each sample was verified by a researcher assessed in a controlled environment with consistent lighting, documented with high-resolution images, and classified by trained personnel from Horticulture department of University of Agriculture, Faisalabad. While direct color evaluation based on sensory panel assessments is a well-established method for determining tomato maturity, our approach leverages a deep learning classification network to capture a broader range of features. In addition to color, the network learns texture, shape, and subtle morphological characteristics that are especially important when color differences are marginal. Furthermore, by training on expert sensory evaluation data, the network internalizes the intricate distinctions that experts use to classify the tomatoes. This results in a model that is robust to variations in illumination and imaging conditions, thereby offering a more comprehensive and reliable solution compared to isolated color analysis. Fig. 2 shows the example tomato image from each class. This classification scheme was designed to capture the nuanced changes in color and ripeness as the tomatoes transitioned through different stages, facilitating the development and validation of a robust maturity indices system, as proposed for this study. To ensure the reliability of the dataset and to avoid data leakage, the splitting of data into training and test sets was

Table 1
Overview of the complete dataset of tomatoes captured under different conditions from different angles.

Tomato Class	Total Tomatoes	Image captured for each tomato	Condition (On/Off)	Total Images	Overall dataset
Green	25	6	Light Without Light	150 + 150	1800
Breaker	25	6	Light Without Light	150 + 150	
Pink	25	6	Light Without Light	150 + 150	
Light Red	25	6	Light Without Light	150 + 150	
Red	25	6	Light Without Light	150 + 150	
Over Mature	25	6	Light Without Light	150 + 150	

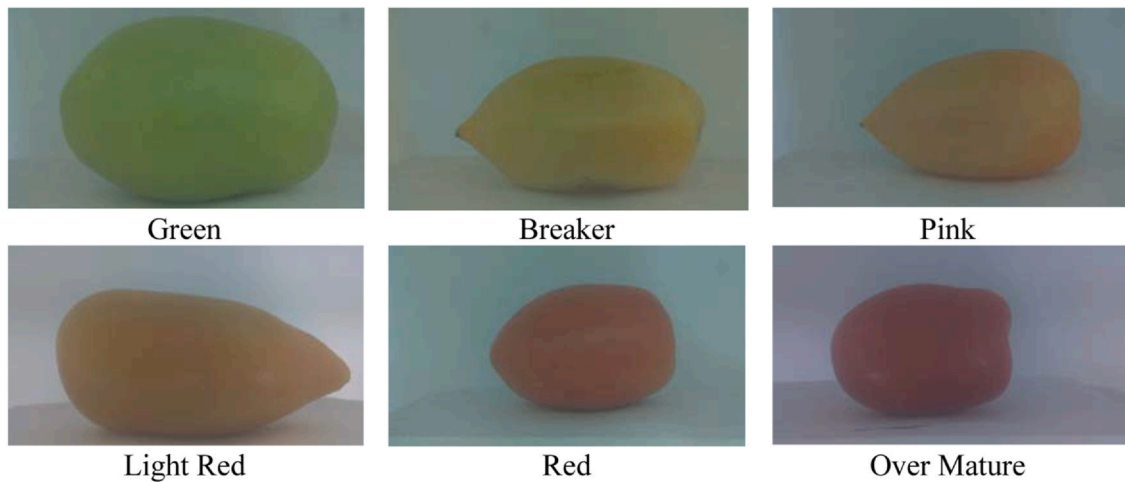


Fig. 2. Different stages of the tomato's dataset.

performed at the tomato-sample level rather than at the image level. That is, all 12 images from a single tomato (captured from six orientations under two lighting conditions) were assigned exclusively to either the training set or the test set. This strategy prevents the overlap of highly similar images between sets and ensures that the model's generalization is tested on truly independent samples.

To acquire the images of tomatoes, an image acquisition system was developed, as shown in Fig. 3(a). This image acquisition system mainly consisted of an imaging chamber, Raspberry Pi Camera v2.1, and NVIDIA Jetson board. The imaging chamber was equipped with a light source to control the illumination. Placing the tomato within this chamber allows for the simulation of different lighting conditions, capturing the tomato's images under both ambient and controlled illumination. This enhances the robustness of the dataset, making it versatile and adaptable to diverse environments and therefore, increases the likely precision of our results. Raspberry Pi Camera v2.1 was integrated with NVIDIA Jetson board through the Camera Serial Interface (CSI) to collect the images. Raspberry Pi Camera v2.1 has an 8MP resolution of 3280×2464 pixels. The image sensor used in this camera was Sony IMX219 which can capture images at 4k resolution. The video capacity of this camera is 1080p, 30fps resolution. For this study, the resolution was set to 1080p (1920×1080 pixels), striking a balance between detail sufficiency and computational efficiency, a decision underscored by the need to process a large dataset while preserving the granularity of the images. While NVIDIA Jetson nano board was used as a central processing unit, a compact yet powerful edge computing platform designed to facilitate deep learning applications in real-time. The Jetson Nano features a quad-core ARM Cortex-A57 processor, an onboard 128-core Maxwell GPU with 2 GB memory.

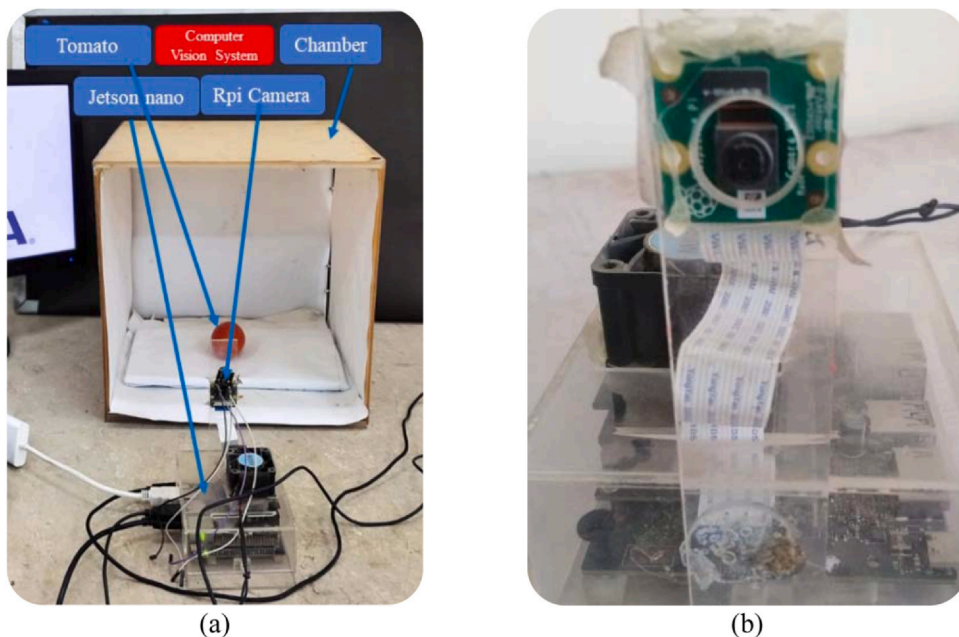


Fig. 3. Computer vision system for this study (a), Front view of integrated RPi camera via CSI with Jetson Nano (b).

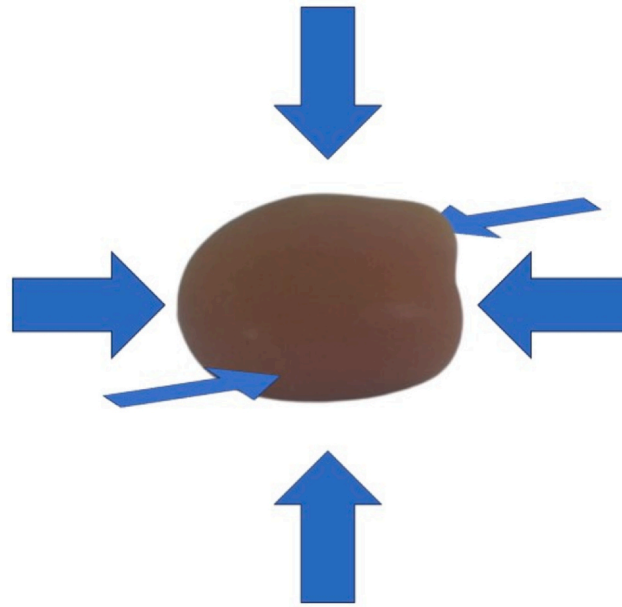


Fig. 4. Illustration of orientation used to capture tomato images.

An overview of the image acquisition system used in our research is illustrated in Fig. 3(b). Each tomato was captured from six distinct orientations: frontal, dorsal (back), up, down, left lateral, and right lateral (Fig. 4), encompassing potential variances in skin texture, color, and other morphological features that are critical for accurate classification. To enhance the authenticity and versatility of our dataset, images were captured under two distinct lighting conditions. The first set of images was acquired in the absence of any external light source, simulating conditions where natural light prevails. The second set involved capturing images with a designated light source, replicating scenarios where additional illumination might influence the visual characteristics of the tomatoes. In total, each tomato underwent the capture of 12 (6 sides, each side in two illumination conditions i.e., Light source turned ON and light source turned OFF) images, resulting in a dataset comprising a total of 1800 images.

2.2. Model architecture

Residual networks, particularly ResNet18 [8], selected as the backbone for this study based on their well-established trade-off between accuracy and computational efficiency, making them particularly suitable for deployment in resource-constrained environments [11,14,16,26]. ResNet18 consists of eighteen layers that utilize these skip connections shown in Fig. 5, allowing information to bypass intermediate layers, thus maintaining data flow and avoiding vanishing gradients. The architecture and building blocks of ResNet18, as illustrated in Table 2, form the foundation of this study due to their robustness and high performance. While

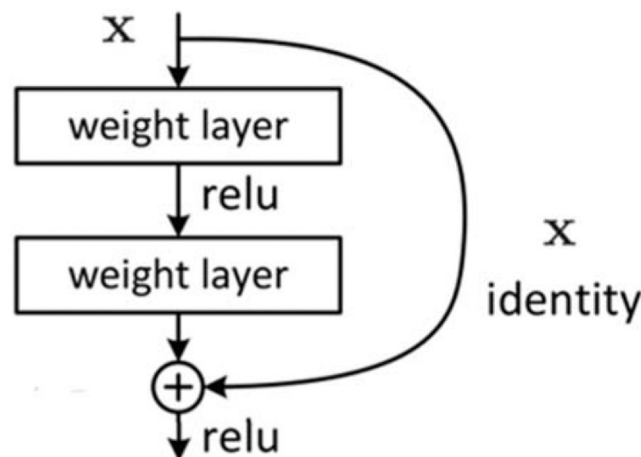


Fig. 5. Illustration of skip connection in a single building block of ResNet model.

Table 2

Architecture and building blocks of base ResNet-18 used in this study.

Layer Name	conv1	conv2 x		conv3 x	conv4 x	conv5 x		FLOPs
Output Size	112 x 112	56 x 56		28 x 28	14 x 14	7 x 7	1 x 1	
18-Layer	7 x 7, 64, stride 2	3 x 3 max pool, stride 2	3 x 3, 64 3 x 3, 64	3 x 3, 128 3 x 3, 128 x 2	3 x 3, 256 3 x 3, 256 x 2	3 x 3, 512 3 x 3, 512 x 2	Average pool, softmax, fc	1.8 x 10 ⁹

alternative lightweight models such as MobileNet and EfficientNet offer compelling advantages, ResNet-18's architecture with its distinctive residual connections proves effective in mitigating vanishing gradient issues and ensuring stable feature extraction, which is especially beneficial for subtle classification tasks like tomato maturity staging. Our methodological contribution centers on a robust multi-stage optimization pipeline, integrating transfer learning, magnitude-based unstructured pruning, and post-training quantization. This practical and systematic approach was explicitly designed to yield a model capable of real-time inference on low-power edge devices. Furthermore, empirical comparisons conducted on our dataset supplemented by literature-reported results (Table 5) demonstrate that the optimized ResNet-18 achieves accuracy and efficiency on par with, or superior to, state-of-the-art models utilizing MobileNet, EfficientNet, and various YOLO versions. Thus, the primary focus of this study is rigorous optimization and deployment of a proven backbone, ensuring robust, scalable, and reproducible performance for agricultural automation applications.

ResNet18 consists of 18 layers, including a 7×7 kernel as the first layer. It comprises four layers of identical ConvNets. Each layer has 2 residual blocks. Each block consists of two weight layers, with a skip connection linked to the output of the second weight layer via a ReLU. ResNet18 has an input size of (224,224,3). The fully Connected (FC) layer is the ultimate output [15]. The base ResNet architecture is shown in Fig. 5, by modifying the final fully connected layer to output six classes, we tailored the network to capture the unique features of each tomato class. This customization enabled the model to achieve high accuracy across all six categories, demonstrating its effectiveness in precise and comprehensive tomato classification. Despite being lightweight, ResNet-18 still has significant computational demands (1.8 billion FLOPs), which can hinder deployment on edge devices with limited resources. To optimize the model for real-world applications for tomato classification, pruning and quantization were employed. Pruning reduces model size and computational load by removing insignificant weights, focusing on the convolutional and fully connected layers. Pruning was primarily applied to the convolutional layers and the fully connected layer. Weight magnitude pruning was employed, where weights below a certain threshold were set to zero. Pruning convolutional layers reduces the number of filters and parameters, directly decreasing computational load. Fully Connected Layer minimizes the number of neurons, further shrinking the model size. After training and pruning, the model underwent quantization. Quantization was applied uniformly across all layers to ensure consistent computational efficiency. Quantizing all layers maximizes the reduction in model size and computational requirements. Lower precision operations are faster and consume less power, which is advantageous for deployment on edge devices. We retained the original 7×7 convolutional layer with stride 2 and padding 3 for the input size (224, 224, 3). The convolutional and residual blocks were pruned and quantized to reduce parameters, and we replaced the original fully connected layer (1000 classes) with a new one for six classes, applying pruning and quantization to it as well.

2.3. Pruning and auto-tuning

Transfer learning with deep networks like ResNet-18 achieves state-of-the-art accuracy, its heavy weight architecture with a large number of parameters and layers requires substantial computational resources during inference. This leads to increased time, power,

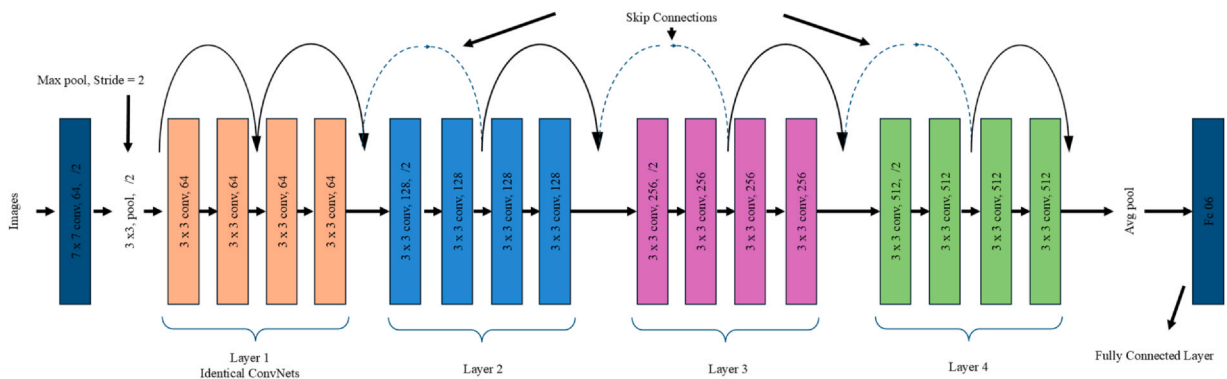


Fig. 6. Detailed overview of the residual network with 18 layers used in this study, highlighting convolutional layers, and building blocks that facilitate deep learning by enabling information.

and memory consumption, making the inference phase expensive and less suitable for deployment on resource-constrained devices. Molchanov et al., [19]. Pruning could help in reducing network complexity by eliminating redundant parameters and connections, thereby decreasing the model's size and computational requirements. Reducing network complexity is crucial for faster inference, improved efficiency on resource-constrained devices like the Jetson Nano, and enhanced generalization by mitigating overfitting. Pruning a CNN model involves decreasing the model's parameters, although this might result in a decline in the model's performance [23]. To speed up inference, pruning allows the final networks to function efficiently even on embedded device i.e., jetson nano.

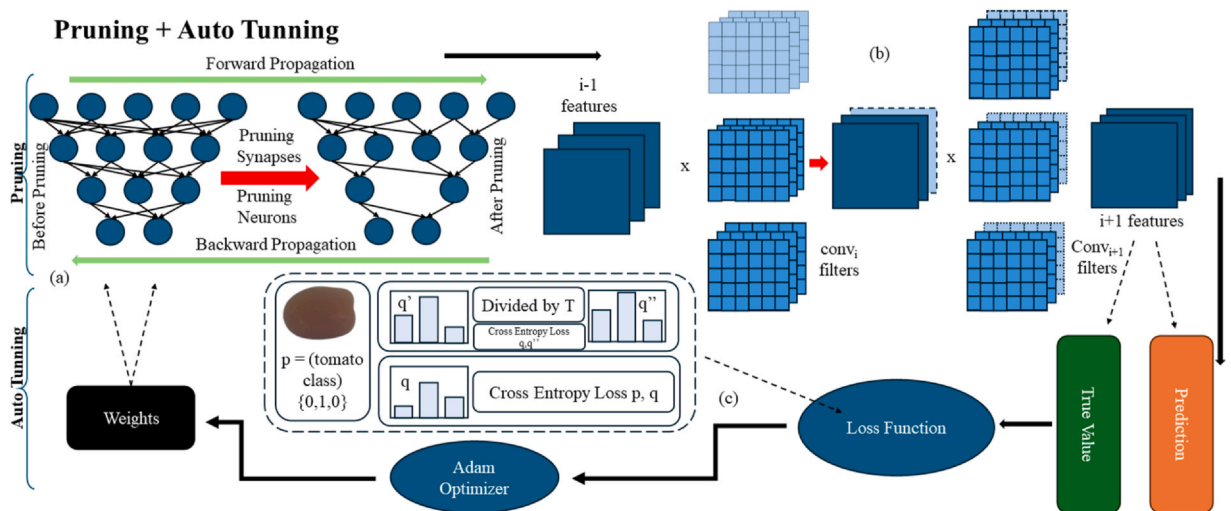
Following the application of transfer learning using ResNet-18, modified to accommodate the six target tomato classes by replacing the original fully connected layer. We implemented magnitude pruning to enhance model efficiency for real-world applications. Magnitude pruning is a model compression technique that eliminates less significant weights based on their magnitudes, thereby reducing computational and memory burdens while maintaining model performance. Specifically, we applied L1 unstructured pruning to the convolutional and linear layers of the network. For each weight θ_i in the set of network parameters are given in Eq. (i).

$$\theta = \{\theta_i\}_{i=1}^n \quad (i)$$

we calculated its absolute value $|\theta_i|$ and pruned weights (Eq. (ii)) that satisfied $|\theta_i| < \tau$ where τ is a threshold determined using quantiles to retain only weights above the 67th percentile:

$$\theta_i^{\text{pruned}} = \{0 \text{ if } |\theta_i| < \tau, \theta_i \text{ if } |\theta_i| \geq \tau. \} \quad (ii)$$

This process resulted in a significant parameter reduction of 66.92 %, decreasing the total number of parameters from 11,179,590 to 3697,762. By sparsifying the network in this manner, we optimized the model for deployment in resource-constrained environments without substantial loss in classification accuracy. Notably, the pruning preserved the structural integrity of ResNet-18's core components, including convolutional layers {Conv2d}, batch normalization layers {BatchNorm2d}, ReLU activations, and {BasicBlock} structures. This preservation ensured that the essential architecture remained intact, allowing the pruned model to maintain its effectiveness. The pruning effectively reduced parameters that contributed minimally to the output, satisfying the condition $L(\theta^{\text{pruned}}) \approx L(\theta)$ for the loss function L . The overall methodology is illustrated in Fig. 7, demonstrating how precise parameter reduction was achieved without compromising performance, thereby making the model highly suitable for efficient, fast processing in practical applications. Upon completion of the pruning procedure, which results in a significant alteration of the network's architectural topology and a consequent shift in its parameter space, a dedicated fine-tuning phase is indispensable for recalibrating the model and recovering performance. For this critical stage, the Adam optimizer (learning rate = 0.001) was implemented, a strategic departure from the Stochastic Gradient Descent (SGD) optimizer utilized during the antecedent full-model training. This transition, while intentionally resetting the momentum states accrued by SGD, was adopted to leverage Adam's distinct advantages in the context of a substantially modified and sparser network. Specifically, pruning induces a new optimization problem; the original parameter landscape is transformed, and the previously optimal convergence trajectory for the dense model no longer relevant for its pruned counterpart. Adam's adaptive moment estimation and per-parameter learning rates are particularly well-suited for navigating such transformed, and potentially more complex, optimization terrains. This allows for rapid exploration and adaptation within the new sparse parameter space, potentially accelerating the convergence to an effective solution for the modified



(a) Unstructured pruning, where synapses—unimportant connections, are pruned to sparse the network. Neurons are also pruned to achieve the same purpose. **(b)** The workflow of L1-based model pruning, in which the light-colored structure is pruned, one filter in conv i is pruned, and its corresponding feature map in layer i will be pruned. Then, the filters in conv $i + 1$ will be adjusted to fit structural changes. **(c)** Fine-tuning is done through an iterative process, and loss is minimized with backpropagation and Adam Optimizer.

Fig. 7. A deep insight into the neural network pruning and auto-tuning process. (a) Pruning while forward and backward propagation, (b) Filters before and after pruning for more efficient feature selection, (c) The cross-entropy loss function by the Adam optimizer to update weights.

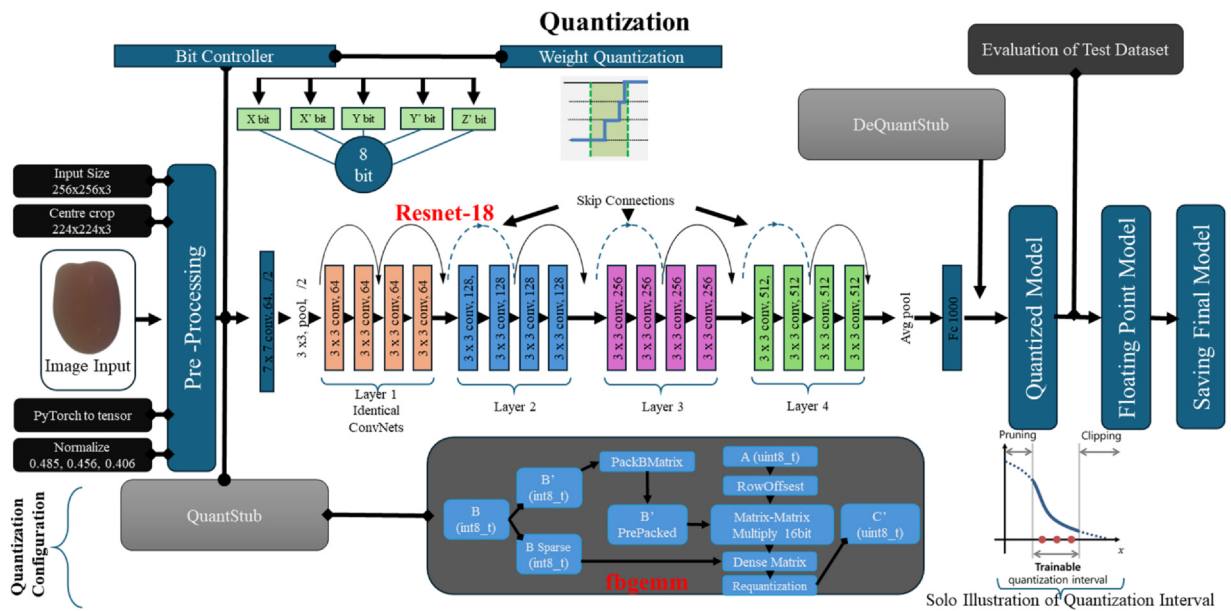


Fig. 8. Implementation of quantization applied to a ResNet-18 architecture.

architecture.

The Pruning + auto-tuning methodology is explained in detail in the appendix below.

2.4. Quantization

In contrast to pruning, where less significant weights are removed to reduce the number of parameters in a neural network, quantization focuses on reducing the precision of the network's weights and activations to lower bit widths also helping to decrease memory and computational requirements. The test dataset was loaded and transformed using **torchvision's ImageFolder**, and a **DataLoader** were created for efficient batching. Quantization and dequantization stubs (**QuantStub** and **DeQuantStub**) were introduced to mark regions for quantization as illustrated in Fig. 8. The quantization methodology and implementation are described in detail in the appendix below.

2.5. Training

The total dataset was divided into training set and a test set, each containing images of tomatoes split into 90:10 (270:30 images) respectively. The validation dataset was further split from the training dataset i.e., 27 images. Importantly, to prevent data leakage, the division of the dataset was conducted by tomato sample. This means that for each unique tomato, all images were kept together in one partition, ensuring that no images from the same sample appear in both the training and test sets. The dataset is organized and loaded using the PyTorch DataLoader, facilitating efficient training and testing. Images undergo a sequence of preprocessing steps to enhance the model's ability to learn relevant features. The transformations include resizing images to 256×256 pixels, center cropping to 224×224 pixels, and normalizing pixel values. The normalization was achieved using the mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$ across the RGB channels. The training process was orchestrated using Stochastic Gradient Descent (SGD) as the optimizer, with a learning rate of 0.001 and momentum of 0.9. The criterion for optimization was the Cross-Entropy Loss. The training loop spans a total of 135 epochs, where the model was iteratively fine-tuned on the training dataset. It was observed that performance began to plateau after approximately 120 epochs, with minimal gain in accuracy and decreasing validation loss, indicating a stable convergence. During training, the model was set to training mode (`resnet18.train()`), and the training dataset was fed through the network in batches of size 4, (increasing batch size on just 2 GB GPU resulted in Fatal Memory Error). The optimizer was used to minimize the computed loss through backpropagation. The training loss was calculated for each epoch, providing insights into the model's learning progress. To prevent overfitting, an early stopping mechanism is incorporated. If the average validation loss does not improve over a set number of consecutive epochs (5 in this study), training was halted to avoid unnecessary model refinement. The subsequent optimization of the ResNet-18 model employed L1-norm unstructured pruning and post-training dynamic quantization. These techniques were deliberately selected for their established efficacy in significantly reducing model parameters and computational overhead with generally minimal impact on accuracy, their relative ease of implementation, and robust support across common deep learning frameworks and hardware. Such characteristics are paramount for practical deployment on resource-limited edge devices like the Jetson Nano, which was a primary objective of this research.

3. Performance metrics

While threshold-based color analysis may appear to be a straightforward approach for classifying tomato maturity based on color, it suffers from several limitations in practical applications. Deep learning models, by contrast, are capable of automatically learning complex features from raw image data. This means that the model can capture not only color information but also texture, shape, and contextual details that are critical for distinguishing between maturity stages, especially when color differences are subtle. In addition, deep learning models have demonstrated robustness to variations in illumination and environmental conditions a frequent challenge in agricultural settings thus offering improved generalizability compared to rigid threshold-based methods. These advantages justify our choice of a deep learning approach for developing a computationally efficient and high-accuracy tomato maturity classification system. Various metrics were employed to assess the models' effectiveness, including classification accuracy, confusion matrix, precision, recall, F1-score, ROC curve, and the Precision-Recall Curves. Classification accuracy is a fundamental metric used to measure the predictive performance of a classification model. It is calculated by dividing the number of correct predictions by the total predictions, as illustrated by Eq. (iii).

$$\text{Accuracy} = \frac{\text{No. of correct predictions}}{\text{Total no. of predictions}} \times 100 \quad (\text{iii})$$

A confusion matrix was drawn to delve deeper into the model's predictive capabilities. This matrix provided a detailed breakdown of the alignment between the model's predictions and the actual ground-truth labels for each class. While not a performance metric itself, the confusion matrix forms the basis for deriving other crucial metrics like precision, recall, and F1-score. Precision evaluated the accuracy of positive predictions, ensuring that positive predictions are reliable and do not falsely classify tomatoes. Eq. (vi) depicts the precision calculation:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (\text{iv})$$

Recall, also known as sensitivity or true positive rate, measured the model's capacity to avoid missing any true maturity class of tomatoes. Eq. (v) represents the recall calculation:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{True Negatives}} \quad (\text{v})$$

The F1-score combines precision and recall into a single metric, emphasizing a balance between correctly identifying tomatoes and not missing any of them. The harmonic mean is employed for this calculation, as shown in Eq. (vi).

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} \quad (\text{vi})$$

The AUC-ROC curve visually represents a classification model's discriminatory power. It plots the True Positive Rate (sensitivity) against the False Positive Rate (1-specificity) across different classification thresholds. An ideal classifier has an AUC-ROC curve with an area of 1, indicating perfect performance, while a random classifier has an area of 0.5, signifying no discriminatory power. In addition to AUC-ROC, Precision-recall curve offers the advantage of comprehensively evaluating a classifier's performance across all threshold levels by plotting the True Positive Rate against the False Positive Rate. It is insensitive to class imbalance and allows for the calculation of the Area Under the Curve (AUC), which summarizes the model's ability to distinguish between classes, making it a powerful tool for comparing and selecting optimal models. Firstly, it is insensitive to class distribution, making it suitable for evaluating models trained on datasets with significant class imbalance. Secondly, it provides a more informative assessment of a classifier's performance when the positive class is of greater interest. Finally, it is particularly useful when the cost of false positives and false negatives varies, allowing us to make informed decisions based on their specific requirements.

4. Results and discussion

4.1. Prediction of tomato maturity

Table 3 and Fig. 9 show the Performance metrics (Accuracy, Precision, Recall, and F1-Score) and confusion metrics for the ResNet model applied in detecting maturity indices of tomatoes, respectively. The evaluation of the three models as transfer learning, pruned + auto-tuned, and quantized showed significant differences in their ability to classify tomato maturity stages accurately. The

Table 3
Performance metrics (Accuracy, Precision, Recall, and F1-Score) for ResNet model applied in detecting maturity indices of tomato.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Transfer Learning	97.78	96.50	96.80	96.65
Pruned + Auto-Tuned	87.96	84.30	85.00	84.65
Quantized	97.81	97.00	97.20	97.10

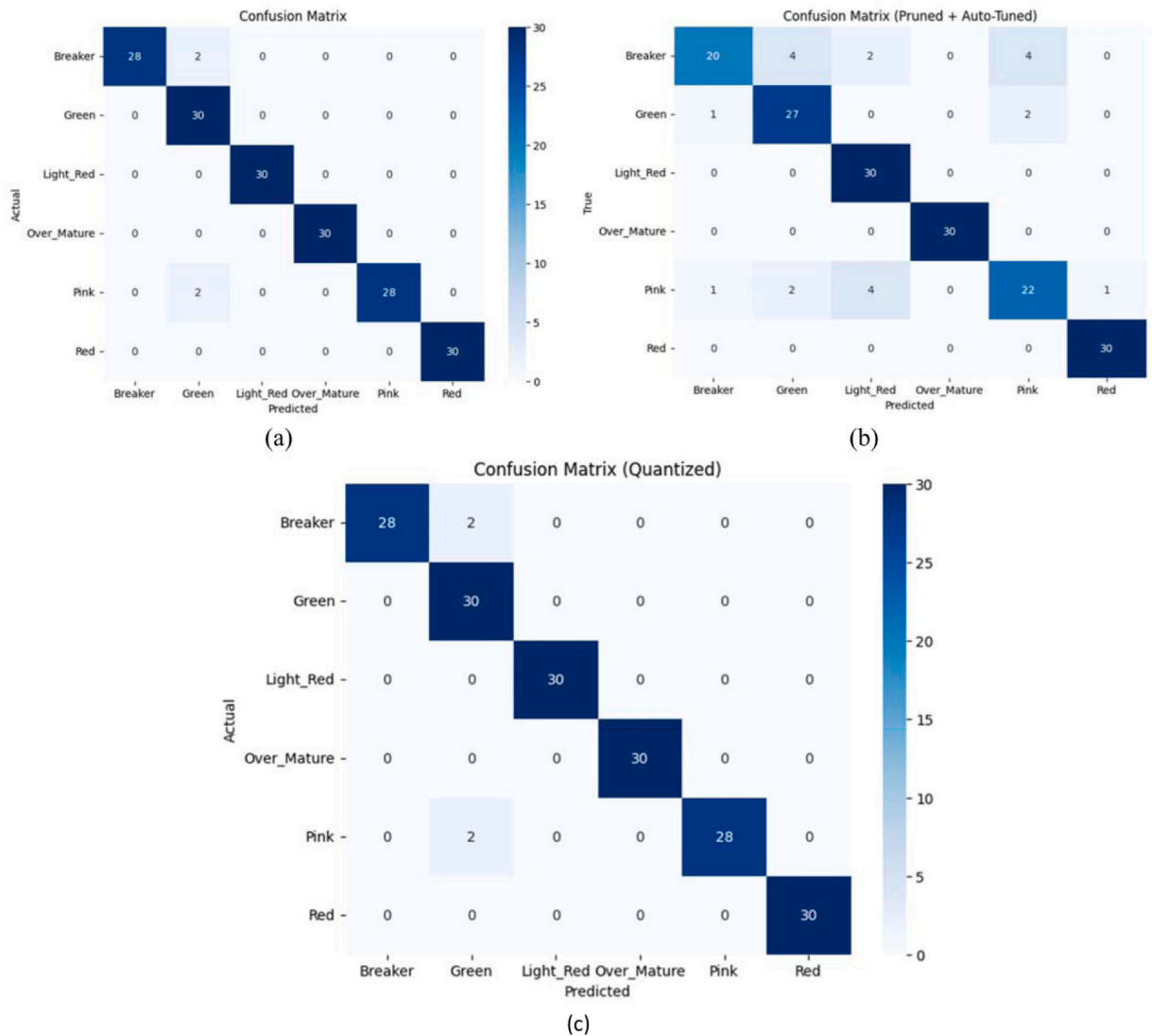


Fig. 9. Confusion metrics for ResNet model representing predicted classes against true classes used for detecting maturity indices of tomato; (a) Transfer Learning model, (b) Pruned + Auto tuned model, (c) Quantized model.

quantized model achieved the highest overall accuracy of 97.81 %, slightly surpassing the transfer learning model's accuracy of 97.78 %, while the pruned + auto-tuned model has little bit lower with an accuracy of 87.96 %. The quantized model demonstrated superior performance across all metrics with an overall accuracy of 97.81 %, precision of 97.00 %, recall of 97.20 %, and an f1-score of 97.10 %, it outperformed the other models. The quantization process reduced the model's computational complexity by using lower-precision representations yet effectively preserved critical feature representations necessary for accurate classification. Since the quantized model originated from the ResNet-18 model, its superior performance was explained by reduced noise in computations due to the lower precision, which acted as a form of regularization, preventing overfitting. By constraining the parameter space, quantization forced the model to focus on key patterns in the data, leading to more efficient classification without compromising accuracy. Additionally, the quantized model executed faster on specialized hardware, enhancing its performance. The model achieved perfect accuracy (100 %) in the green, light red, red, and over-mature stages, indicating exceptional precision and recall in these categories. In the more challenging breaker and pink stages, it maintained a high accuracy of 93.33 % each, with minimal misclassifications, primarily in the green stage. The quantized model's superior performance is attributed to its ability to maintain high accuracy while reducing model size and computational demands. The quantization process compresses the model by using lower-precision data types, enhancing efficiency and making it highly suitable for real-time applications in resource-constrained environments, such as agricultural fields. The high precision and recall values indicate that the model consistently identifies true positives while minimizing false positives across all maturity stages. The high results of the quantized model over the transfer learning model in accuracy, precision, recall, and f1-score become significant in practical scenarios. This efficiency does not come at the expense of accuracy, making the quantized model the optimal choice for applications requiring both speed and precision. The

transfer learning model, while closely following the quantized model, had a slightly lower overall accuracy of 97.78 %, precision of 96.50 %, recall of 96.80 %, and an f1-score of 96.65 %. The model also achieved 100 % accuracy in the green, light red, red, and over-mature stages. However, in the breaker and pink stages, the accuracy was slightly lower at 93.33 %, with misclassifications in the green stage. The minimal difference in performance compared to the quantized model becomes significant when considering computational efficiency for practical deployment. The transfer learning model, although highly accurate, does not offer the same computational benefits. Transfer learning models, derived from pre-trained models on large datasets, offer a substantial number of parameters that continue to require higher computational resources and memory, limiting efficiency gains. Its slightly lower precision and recall suggest a marginally higher misclassification rate than the quantized model. This model remains a viable option in environments with abundant computational resources due to its strong performance. However, it is less advantageous for applications where efficiency is paramount than the quantized model. The Pruned + auto-tuned model exhibited the lowest performance, with an overall accuracy of 87.96 %, precision of 84.30 %, recall of 85.00 %, and an f1-score of 84.65 %. Despite achieving 100 % accuracy in the light red, red, and over-mature stages, its performance in the breaker, pink, and green stages was considerably lower. The accuracies were 66.67 % for breaker, 73.33 % for pink, and 90 % for green. The pruning process, intended to eliminate redundant parameters, inadvertently removed essential features, leading to increased misclassifications and a decline in both precision and recall. The pruned + auto-tuned model highlights the risks associated with aggressive model optimization techniques. Pruning reduced the model's ability to distinguish between maturity stages with subtle differences, such as breaker and pink stages. The significant drop in accuracy, precision, recall, and f1-score indicates that essential features were removed, compromising the model's generalization capability. This underperformance renders it less suitable for tasks where accurate classification is critical.

The Receiver Operating Characteristic (ROC) curve analysis revealed that the Quantized Model consistently outperformed the other models across all tomato maturity stages. For the Quantized Model, the Breaker and Pink stages achieved an Area Under the Curve (AUC) of 0.99, indicating near-perfect sensitivity and specificity. The Green, Light Red, Red, and Over-Mature stages each achieved a perfect AUC of 1.00, reflecting the model's exceptional ability to distinguish these stages without error. This superior performance suggests that the quantization process effectively preserved the model's discriminative power, enabling it to maintain high true positive rates while minimizing false positives. The Precision-Recall (PR) curve analysis further substantiated the Quantized Model's superior performance. The Quantized Model achieved high average precision scores across all maturity stages, with perfect scores of 1.00 for the Light Red, Red, and Over-Mature stages. For the Breaker and Pink stages, the average precision was 0.95 and 0.93, respectively. These high precision levels, even at increased recall values, indicate the model's robustness in correctly identifying true positives while minimizing false positives. The stable and high PR curves suggest that the Quantized Model effectively handles the trade-off between precision and recall, maintaining consistent performance across different thresholds. The complete illustration of ROC and PR curves are given in [Figs. 10 and 11](#) respectively.

In comparison, the Transfer Learning Model also demonstrated strong performance, with AUCs of 0.99 for the Breaker and Pink stages and 1.00 for the remaining stages. While the ROC curves for both the Quantized and Transfer Learning models were similar, the Quantized Model's marginally higher overall accuracy underscores its enhanced classification capabilities after quantization. The Pruned + Auto-Tuned Model, however, exhibited diminished performance, particularly in the Breaker and Pink stages, with AUCs of 0.93 and 0.95, respectively. These lower AUC values indicate challenges in the model's ability to maintain low false positive rates, likely due to the loss of critical discriminative features during the pruning process. The Transfer Learning Model exhibited similar trends in the PR curves, with average precision scores of 0.95 for the Breaker stage and 0.93 for the Pink stage. Although these scores are commendable, the slight edge of the Quantized Model highlights its enhanced ability to retain critical feature representations after quantization. The Pruned + Auto-Tuned Model showed significantly lower average precision scores of 0.79 for the Breaker stage and 0.80 for the Pink stage. The substantial drop in precision as recall increased indicates a higher rate of false positives, reflecting the model's struggle to distinguish between stages with subtle differences due to the pruning process.

The analyses of both ROC and PR curves underscore the Quantized Model's ability to maintain high classification performance while reducing computational complexity. The quantization process not only compressed the model but also preserved essential discriminative features necessary for accurate classification. This is particularly advantageous in agricultural applications where computational resources are limited, and rapid, precise classification is essential for tasks such as automated harvesting and sorting. The Quantized Model's superior performance over the Transfer Learning Model, albeit numerically slight, becomes significant in practical deployments that demand both efficiency and accuracy. In contrast, the diminished performance of the Pruned + Auto-Tuned Model highlights the potential drawbacks of aggressive model optimization techniques. The pruning inadvertently eliminated important features crucial for distinguishing between maturity stages with subtle visual differences, leading to decreased sensitivity and precision, especially in the Breaker and Pink stages. This outcome emphasizes the importance of carefully balancing model simplification with the preservation of critical feature representations to ensure reliable performance.

The results confirm that the Quantized Model's ROC and PR curve analyses offer the best balance between accuracy and computational efficiency among the evaluated models. Its exceptional ability to accurately classify all tomato maturity stages, combined with reduced model size and increased inference speed, makes it highly suitable for integration into edge devices platforms. The Quantized Model's performance demonstrates that quantization is an effective strategy for optimizing deep learning models for real-world applications without sacrificing accuracy. Future research could explore further enhancements to the quantization process and investigate hybrid optimization techniques that combine quantization with selective pruning to achieve even greater efficiency while maintaining high classification performance.

Beyond accuracy, it is essential to evaluate the statistical robustness of the classification models to validate their predictive reliability. To quantify this robustness, 95 % confidence intervals (CIs) were calculated for the accuracy metrics of all three developed models. These confidence intervals provide an estimation of the range within which the true accuracy of the model lies, reflecting the uncertainty inherent in model predictions due to sampling variability.

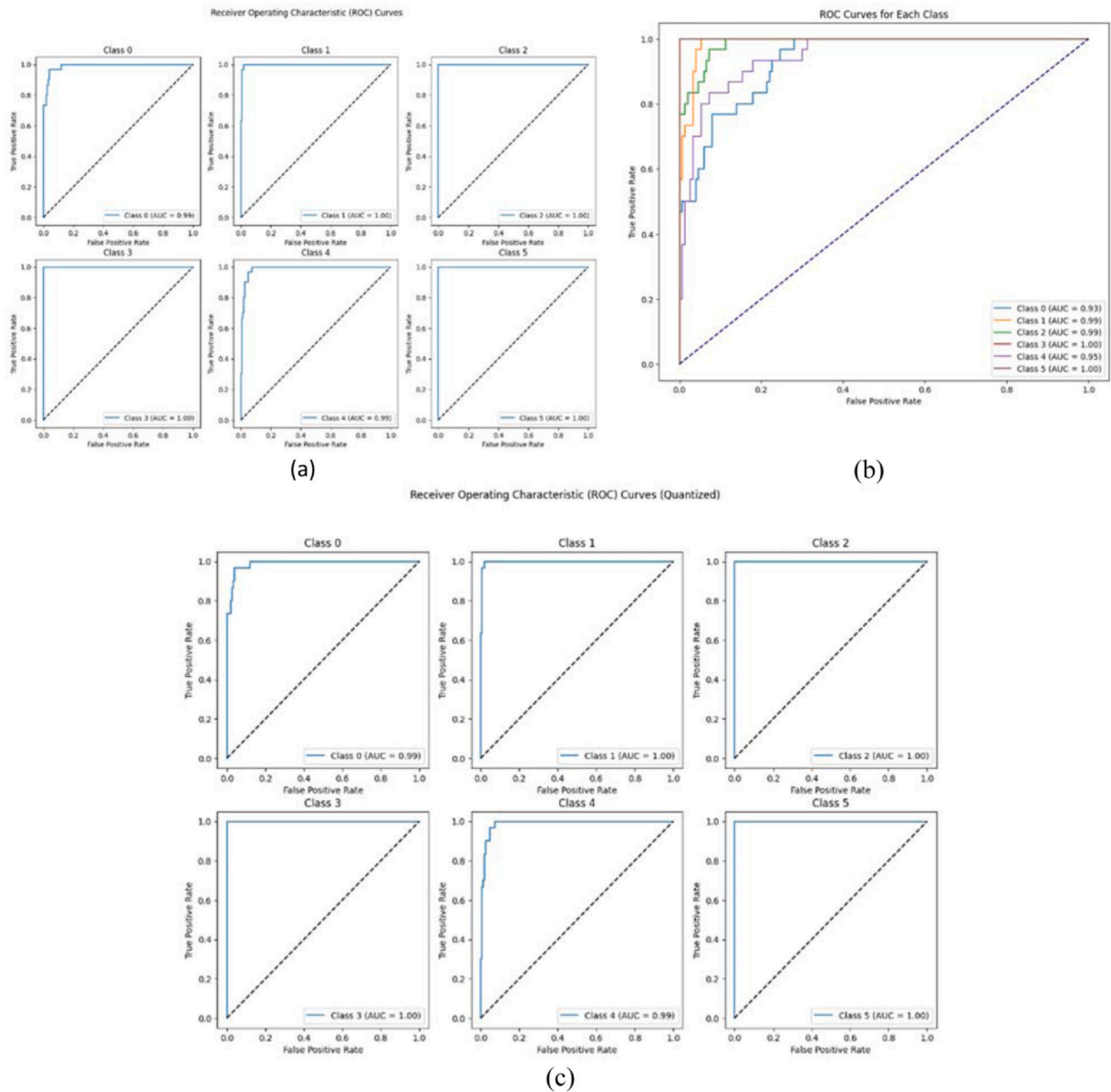


Fig. 10. ROC (Receiver Operating Characteristic) curves illustrating the sensitivity (true positive rate) versus the specificity (false positive rate) of the ResNet model (a) Transfer Learning model, (b) Pruned + Auto tuned model, (c) Quantized model used for detecting tomato maturity indices.

The 95 % CI for accuracy was calculated using the standard formula for binomial proportion confidence intervals:

$$CI = \hat{p} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

where $\hat{p}$ is the observed accuracy, n is the number of test samples, and z is the z-score corresponding to the 95 % confidence level ($z = 1.96$).

The calculated confidence intervals illustrated in Fig. 12 provide crucial insight into the reliability and generalizability of our classification models. For both the Quantized Model (97.81 % accuracy; CI: [95.50 %, 99.20 %]) and the Transfer Learning Model (97.78 % accuracy; CI: [95.45 %, 99.15 %]), the narrow and highly overlapping intervals indicate that these models consistently achieve high performance across repeated samples. This high level of statistical certainty suggests that the reported accuracy is not a product of random chance or sampling artifacts but reflects genuine, reproducible model capability. In practical terms, these findings imply that both models are robust choices for real-world deployment, with little risk of significant performance drop under similar operational conditions. The Pruned + Auto-Tuned Model (87.96 % accuracy; CI: [83.45 %, 91.35 %]) exhibits a noticeably wider confidence interval, reflecting greater variability and uncertainty in predictive performance. This suggests that while optimization

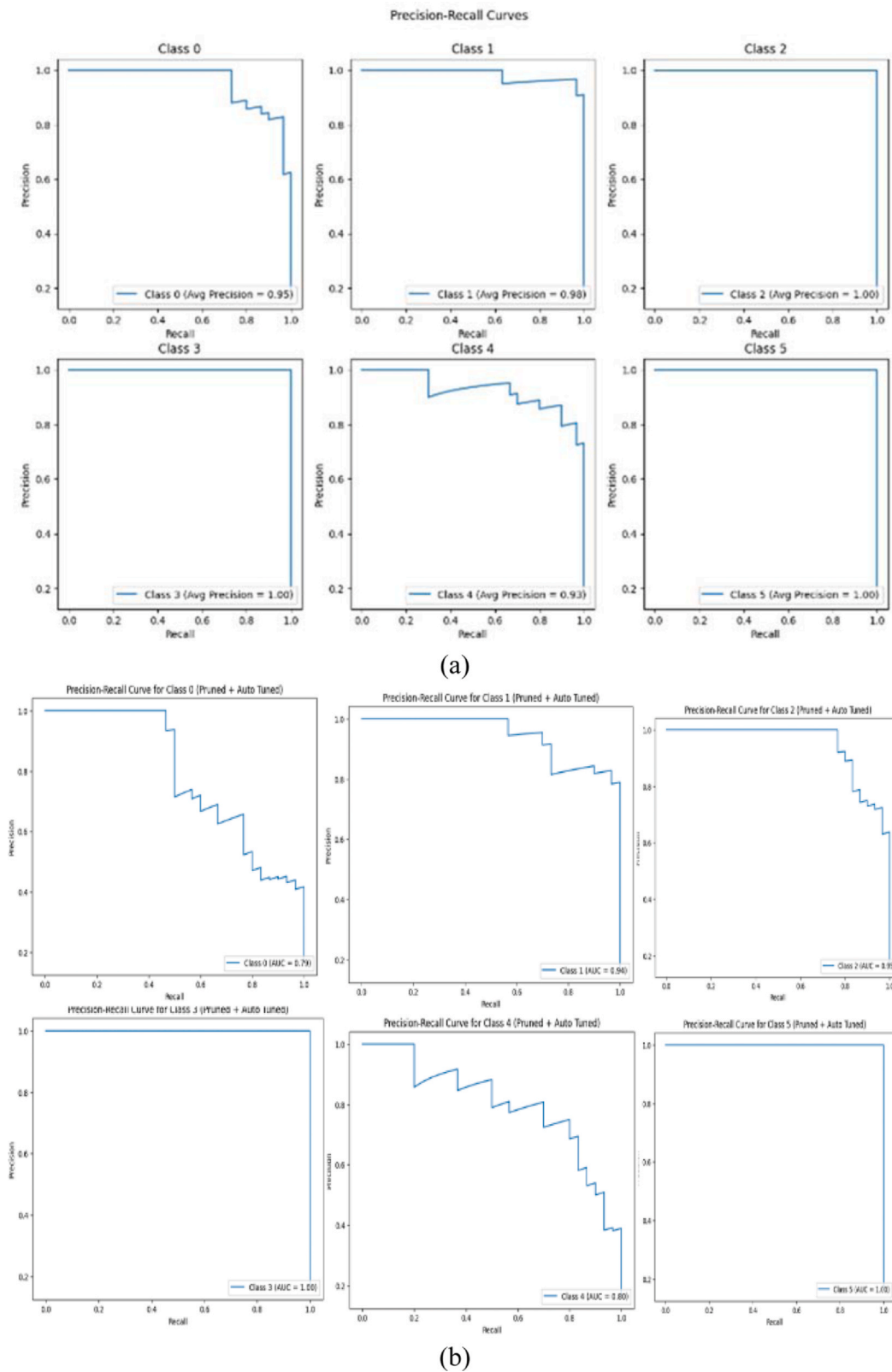


Fig. 11. Precision, recall curve of the ResNet model (a) Transfer Learning model, (b) Pruned + Auto tuned model, (c) Quantized model used for detecting tomato maturity indices.

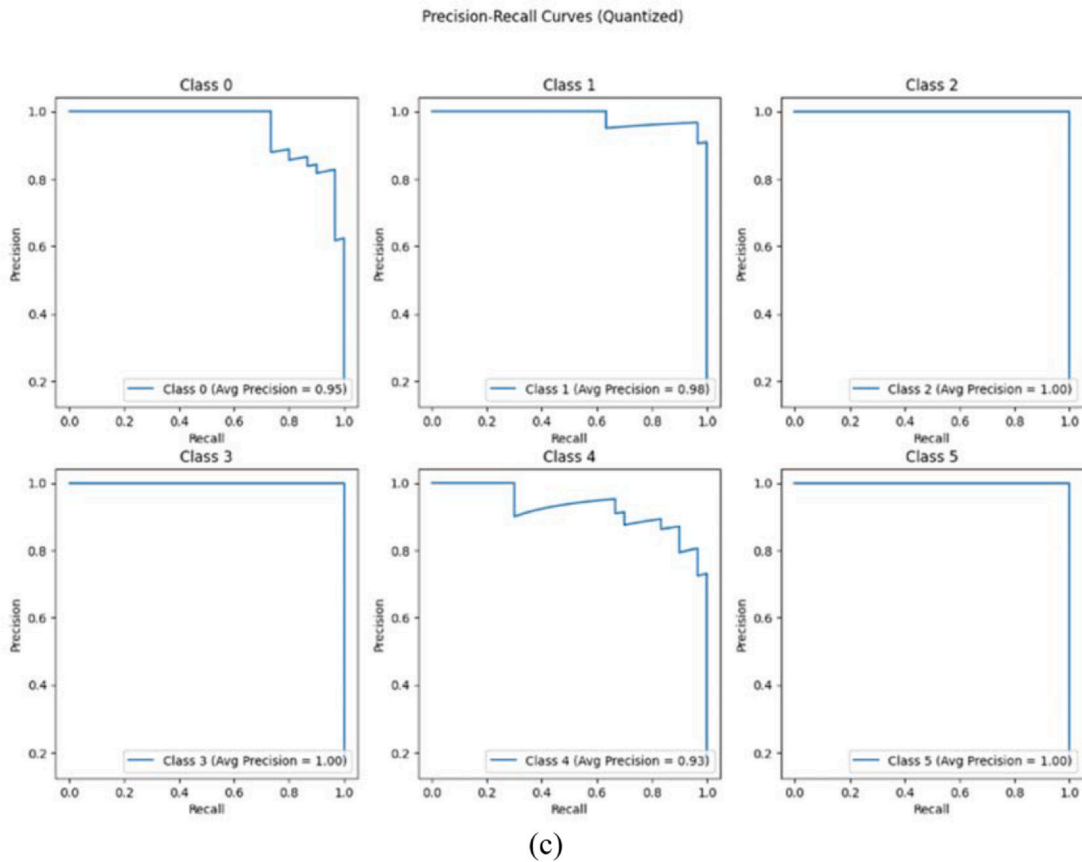


Fig. 11. (continued)

techniques such as pruning and auto-tuning can enhance computational efficiency, they may also introduce instability or reduce reliability, particularly when applied aggressively. For stakeholders considering model deployment in critical agricultural settings, these statistical results highlight the trade-off between efficiency and reliability and reinforce the importance of validating model performance with rigorous statistical measures.

Representative of correctly classified tomato maturity stages are shown in Fig. 13, highlighting the model's ability to distinguish subtle differences across the full spectrum of ripeness. These visualizations complement the quantitative results by illustrating real predictions on test samples

4.2. Deployment of models on jetson nano

After the models' development and validation stages, their deployment on the Jetson Nano platform provides crucial insights into their operational efficiency and performance under real-world conditions. This section deals with the specifics of deploying the transfer learning, pruned + auto-tuned, and quantized models on the Jetson Nano, evaluating key metrics such as build time, model load time, CPU and GPU usage, maximum processing unit temperature, and average classification time as illustrated in Table 4.

The transfer learning model exhibited the longest build time of 3 min and 40 s and a model load time of 74 s, reflecting its complexity and larger size. Its average cpu and gpu usage during inference was high at 80 %, and it reached a maximum temperature of 54°C. The average classification time was the slowest among the models at 0.01201 s per image. These results indicate that while the transfer learning model is highly accurate, its substantial computational demands make it less suitable for deployment on resource-constrained devices like the jetson nano. In contrast, the pruned + auto-tuned model demonstrated improved deployment efficiency with a reduced build time of 1 min and 22 s and a model load time of 40 s. The cpu and gpu usage decreased to 70 %, and the maximum temperature lowered to 52°C. Notably, the average classification time significantly improved to 0.001073 s per image. However, this efficiency gain came at the cost of reduced accuracy and precision in certain maturity stages, suggesting that critical features may have been lost during the pruning process. This trade-off highlights the challenges of aggressive model optimization techniques that may compromise essential predictive capabilities. The quantized model achieved the best balance between performance and efficiency. With a build time of 1 min and 56 s and a model load time of 56 s, it offered moderate initialization periods. The model's cpu and gpu usage was the lowest at 65 %, and it maintained the lowest maximum temperature of 51°C, indicating optimal resource utilization. Impressively, it achieved the fastest average classification time of 0.000975 s per image while

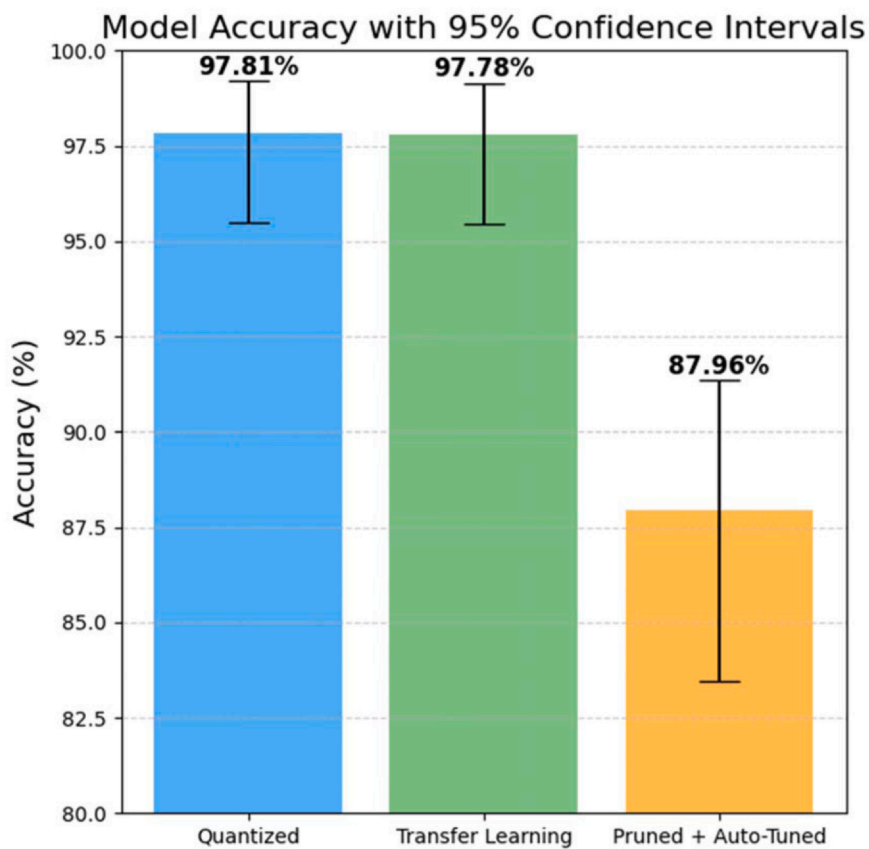


Fig. 12. Illustration of model accuracy with 95 % Confidence Interval.

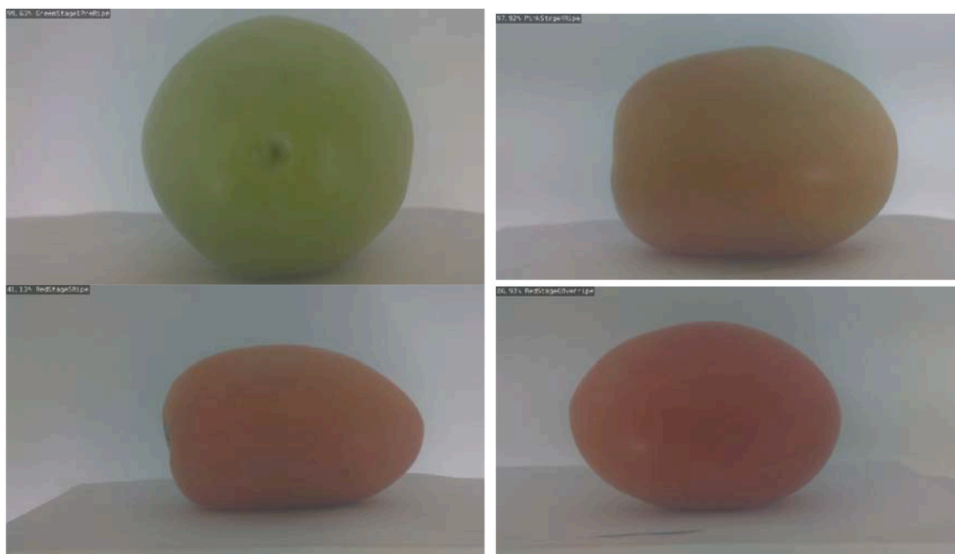


Fig. 13. Illustration of correctly identified class of tomato maturity.

maintaining high accuracy across all maturity stages. The reduced computational complexity translates to faster inference times and lower power consumption, which are critical factors for deployment on edge devices and mobile platforms. These findings demonstrate that quantization effectively reduces computational demands without sacrificing model performance, making it highly suitable for deployment on edge devices like the jetson nano.

Table 4
Deployment Metrics of Models on Jetson Nano.

Metric	Transfer Learning	Pruned + Auto-Tuned	Quantized
Build Time (minutes:seconds)	3:40	1:22	1:56
Model Load Time (seconds)	74	40	56
CPU and GPU Usage (%)	80	70	65
Maximum Temperature (°C)	54	52	51
Average Classification Time (s/image)	0.01201	0.001073	0.000975

The deployment results have significant implications for the practical application of deep learning models in resource-constrained environments. The quantized model's superior operational efficiency and rapid inference times make it ideal for real-world agricultural applications where quick and accurate classification is critical, such as automated harvesting and sorting systems. Its ability to maintain high accuracy with reduced computational requirements suggests that quantization is an effective strategy for optimizing models for edge deployment. The observed drawbacks of the pruned + auto-tuned model emphasize the need for cautious application of pruning techniques. While pruning can enhance efficiency, it may inadvertently remove essential features necessary for accurate classification, particularly in stages with subtle differences. Future research should focus on developing advanced pruning methods that preserve critical features or exploring hybrid optimization approaches that combine pruning with quantization to balance efficiency and accuracy. Moreover, real-world testing of the quantized model on the Jetson Nano in operational agricultural settings would provide valuable insights into its performance under practical conditions. Such evaluations could inform further refinements to the model and optimization processes, ensuring robustness and reliability in diverse environments. Expanding the scope of deployment studies to include other edge devices and exploring the impact of varying environmental factors on model performance would also be beneficial.

This study's comparative analysis of the Transfer Learning, Pruned + Auto-Tuned, and Quantized models for classifying tomato maturity stages underscores the superior performance of the Quantized Model. The Quantized Model not only achieved the highest accuracy, precision, recall, and F1-score but also demonstrated exceptional performance in ROC and PR curve analyses. Its ability to maintain perfect classification in most maturity stages, coupled with high AUC values, highlights its robust discriminative power. The efficiency gains from quantization are particularly noteworthy. By reducing the model's computational complexity through lower-precision data representations, the Quantized Model achieved faster inference times and reduced memory usage without compromising accuracy. This makes it highly suitable for deployment on resource-constrained edge devices like the NVIDIA Jetson Nano, which are commonly used in agricultural settings for real-time monitoring and automated decision-making. In contrast, the Transfer Learning Model, while highly accurate, does not offer the same level of computational efficiency. Its higher resource and memory requirements make it less ideal for deployment in environments with limited computational capabilities. However, it remains a viable option in scenarios where computational resources are ample and the slight decrease in efficiency is acceptable given its strong performance. The Pruned + Auto-Tuned Model's diminished performance highlights the risks associated with aggressive model optimization techniques. Pruning, intended to eliminate redundant parameters, inadvertently removed critical features necessary for accurate classification, particularly in stages with subtle visual differences like Breaker and Pink. This resulted in a significant drop in accuracy, precision, recall, and F1-score, rendering the model less suitable for tasks where precise classification is critical.

This study employed a quantized model for tomato maturity detection under controlled conditions, demonstrating excellent performance. However, for real-world agricultural applications, future studies must address key challenges related to environmental variability. A diverse dataset incorporating fluctuating lighting, varying backgrounds, and other real-world complexities will be critical to ensure accurate and robust performance in uncontrolled environments. Deep learning models trained on such diverse datasets will better reflect the conditions encountered in field applications. Evaluating the performance of this quantized model across a range of edge devices, each with unique computational capabilities can bring more robustness. Expanding this work to different crop varieties and physiological traits is another area for future exploration. The ability of the quantized model to operate on low-power edge devices with minimal resource consumption paves the way for widespread adoption, particularly among small and medium-sized farms. By providing efficient, real-time analysis of crop maturity, the results from this study could be used for timely decisions regarding harvest timing and disease management, ultimately improving yield and reducing crop losses. Moreover, the findings from this study could be scaled up for use in large-scale farming operations, facilitating automated sorting and grading of agricultural produce through industrial conveyors and on-farm systems in warehouses and processing facilities. As agricultural systems become increasingly data-driven, cutting-edge computing and AI will be indispensable in providing real-time insights directly from the field. However, issues such as data privacy and security will become critical beyond performance and scalability. Since edge devices process data locally, this approach enhances data security by reducing transmission to centralized servers. In addition, lightweight models optimized for edge devices can further streamline data collection and analysis processes, minimizing cloud-based system dependency that requires more resources. The comparative analysis of recent existing studies with our is illustrated in [Table 5](#).

5. Limitations and future directions

While our proposed tomato maturity classification model demonstrates promise in efficiently classifying maturity stages in low-power edge devices, few limitations are there. Our study utilized 25 unique tomato samples per maturity stage, and although capturing multiple perspectives helped enrich the feature set, this relatively small number of unique samples may constrain the generalizability of the results; future work should involve larger and more diverse datasets to fully capture the variability encountered in real-world agricultural settings. Moreover, the controlled image acquisition conditions characterized by consistent

Table 5

Comparative analysis of recent studies.

Model	Approach	Accuracy %	Draw back	Reference
Quantized ResNet	Our Study	97.81	-	-
Transfer Learning	Our Study	97.78	High computational resources required	
Pruned + Auto-Tuned ResNet	Our Study	87.96	Loss of essential features due to aggressive pruning	
YOLOv5s-tomato	Improved YOLOv5 Model	97.42	Struggle with small-target tomatoes	(R. [13])
MHSA-YOLOv8	MHSA-YOLOv8 Model	86.42	Increased model size due to MHSA integration	(P. [12])
MTS-YOLO	MTS-YOLO Model	92.0	Require optimization for deployment	Wu et al., [35]
SE-YOLOv3-MobileNetV1	SE-YOLOv3-MobileNetV1 Network	95.0	Degrade in natural greenhouse environments	Su et al., [31]
EfficientNetB0	Transfer Learning Approach	90.0	Require large datasets for optimal performance	Lumoring et al., [17]
YOLOv9	YOLOv9 Model	92.49	Potential overfitting	Vo et al., [33]

lighting and uniform backgrounds do not fully reflect the environmental challenges such as variable illumination, diverse backgrounds, or occlusions that are typical in field conditions, potentially impacting the model's performance when deployed in field instead of industry. The reduction in numerical precision could make the quantized model more sensitive to extreme real-world variabilities not extensively covered in the current dataset. Partial occlusions by leaves or other objects, or highly cluttered backgrounds common in fields, introduce noise and ambiguity. The model might limit its ability to effectively isolate tomato features from such complex visual information, potentially leading to a greater drop in performance compared to its full-precision counterpart. Furthermore, even though our optimized model shows promising real-time performance on a specific low-power edge device, the deployment was tested under limited hardware configurations and evaluation scenarios, meaning that variations in hardware capabilities or integration challenges could alter its effectiveness across different agricultural settings. Finally, while quantization and pruning techniques significantly reduced computational demands without a substantial loss in accuracy, these optimization methods might introduce trade-offs that affect the model's ability to discriminate subtle differences in maturity levels under challenging conditions, indicating a need for further research to understand their long-term impact on model reliability across broader application scenarios. Additionally, investigating hybrid optimization techniques such as combining model quantization with selective pruning (structured, mixed-precision, QAT) could enhance efficiency while maintaining critical features for accurate classification. Extending the quantized model framework to other crops and maturity indices offers a promising avenue to assess its versatility across various agricultural products. Finally, the development of user-friendly deployment frameworks would facilitate seamless integration of the quantized model into existing agricultural workflows, increasing its accessibility for farmers and agricultural professionals.

6. Conclusion

The primary goal of this research was to develop a computationally efficient tomato classification model using the ResNet-18 architecture. This study aims to address the challenges posed by computationally intensive deep learning networks through pruning and quantization approaches, thereby enabling their deployment in resource-constrained agricultural environments. The quantized model emerged as the best option for deployment on the Jetson Nano, achieving an accuracy of 97.81 % and the lowest classification time of 0.000975 s per image. It also demonstrated the most efficient resource usage, with 65 % CPU and GPU utilization, making it suitable for real-world applications in agriculture. The pruned and auto-tuned model showed improved deployment efficiency but required careful balancing to maintain performance, with an accuracy of 87.96 %. These findings emphasize the importance of optimizing both computational efficiency and accuracy for edge device deployment. Future work will focus on extending these techniques to other crops and agricultural processes to enhance productivity and sustainability in diverse environments.

CRediT authorship contribution statement

Muhammad Waseem: Writing – original draft, Methodology, Investigation, Conceptualization. **Muhammad Muzzammil Sajjad:** Writing – original draft, Validation, Formal analysis. **Laraib Haider Naqvi:** Visualization, Validation, Data curation. **Yaqoob Majeed:** Writing – review & editing, Supervision, Project administration. **Tanzeel Ur Rehman:** Writing – review & editing, Validation, Software, Investigation. **Tayyaba Nadeem:** Writing – review & editing, Resources

Data availability

The dataset for the current study will be available from the corresponding author on request until it is published online in the near future to enhance reproducibility and contribute to the broader research community.

Declaration of Competing Interest

All authors have no conflict of interest.

Acknowledgement

This research was supported by National Research Program for Universities, Higher Education Commission (HEC), Pakistan (20–15545/NRPU/RandD/HEC/2021 2021) and University of Wyoming, USA. We thank **Chung-Hsuan Huang** from Department of Electronics and Electrical Engineering, National Yang-Ming Chiao Tung University for continuous support during research period.

Appendix

Pruning Methodology:

The process of pruning is shown in the above horizontal row of the Fig. 7, and the below row follows auto-tuning, and their interlink is illustrated. The whole process is divided further not three sections as (a), (b), and (c). The section (a) illustrates unstructured pruning which involves the removal of individual synapses, or connections, in a neural network without any specific pattern or structure. This was done based on the magnitude of the weights. The identification and elimination of connections with low-importance aims was made to reduce the overall size of the network. In this study, a simple feedforward network with multiple layers was used. Each connection between neurons had a corresponding weight. During unstructured pruning, we analyzed the weights of these connections, and connections with weights below a 66 % threshold are pruned, effectively removed them. Section (b) illustrates the workflow of pruning L1-unstructured with the context of feature map removal. In the initialization phase of this study, a neural network model undergoes training using standard techniques without any pruning, featuring multiple layers with filters or neurons. Following this, the L1 of each parameter within the network is computed, representing the sum of the absolute values of its elements, where i ranges from 1 to L . $W^{(i)}$ represents the connection weight matrix for the i -th layer shown in Equation (iii), with M_i as the number of input channels, N_i as the number of output channels, and $K \times K$ as the kernel dimensions.

$$\{W^{(i)} \in \mathbb{R}^{(M_i \times N_i \times K \times K)}, 1 \leq i \leq L\} \quad (\text{iii})$$

Parameters possessing smaller L1-norm values, indicative of lesser importance to overall model performance, were identified and earmarked for potential pruning. Subsequently, parameters with L1 values falling below 66 % threshold undergo pruning, entailing the removal of entire filters or channels in convolutional layers. This pruning process induces a structural change, necessitating adjustments in the filters of the subsequent convolutional layer to maintain network architecture and connectivity. The adjustment involved redistributing weights or reassigning connections. This was done by fine-tuning to further enhance or restore its performance. This iterative process of L1-based model pruning ensures network size reduction while preserving or improving its functionality.

Section (c) illustrates the auto-tuning or fine-tuning of the pruned model, particularly focused on minimizing the loss and increasing accuracy. In this study, a ResNet-18 model previously pruned through magnitude pruning was loaded. The dataset is then split into training and validation sets, and the model is trained using the training set while monitoring its performance on the validation set. The Adam optimizer is employed with a learning rate of 0.001, and the CrossEntropyLoss is used as the loss function. The training is performed over several epochs, and for each epoch, both training and validation losses and accuracy were monitored. The fine-tuned model is subsequently saved. Fine-tuning allowed the pruned model to adapt to the specifics of the new dataset, potentially enhancing its performance compared to the original pruned model. In this study, the loss function is minimized as the input image is labeled as belonging to the Tomato maturity class, with the probability expressed as $\{0, 1, 0\}$. Following the inference of both the pruned and fine-tuned networks, the algorithms produce classification results q and q' . The image exhibits traits associated with close maturity class that are not distinctly reflected in q and q' . emerges, resulting in a classification result q'' that provides more intricate insights. By training the pruned network with fine-tuning, the final network enhances its accuracy based on the distinctive characteristics acquired during the pruning process.

Quantization Methodology

In this study, during the initial configuration phase, the classification task is defined with 6 classes, with a specified directory for saving the quantized model. The pre-trained ResNet-18 model was loaded without its final fully connected layer, which was then adjusted to accommodate the new task's 6 classes. The model was loaded with weights from a previously trained model (transfer learning), set to evaluation mode, and path for test data and data transformations were defined. Quantization and dequantization are essential stages in converting a model's weights and activations from floating-point precision to lower bit precision, like int8, aiming to reduce memory storage demands and enhance computational efficiency during inference. These steps are particularly crucial for deploying models on resource-constrained devices. In addition to reducing the memory and computational demands of the model, quantization yields improvements in classification accuracy. One reason for this improvement is that quantization introduces a form of regularization by limiting the precision of the model's weights and activations. This constraint prevents the model from overfitting to noise in the training data, thereby enhancing generalization performance. Moreover, the nature of our image dataset where images are captured under lighting and from multiple orientations ensures that the essential features are retained even when parameter precision is reduced. This enables the quantized model to effectively extract and utilize discriminative features, contributing to both its efficiency and accuracy. The QuantStub identified the input tensor region where quantization was applied, serving as the starting point for quantizing the input Equation iv, v vi respectively. X be the input tensor to the model, $Q(X)$ be the quantized tensor, and $D(Q(X))$ be the dequantized tensor.

$$Q(X) = \text{QuantStub}(X) \quad (\text{iv})$$

On the other hand, the DeQuantStub marked the output tensor region where dequantization occurred, signifying the end of the quantized region where the tensor is converted back to floating-point precision.

$$D(Q(X)) = \text{DeQuantStub}(Q(X)) \quad (\text{v})$$

The quantization configuration is set using the 'fbgemm' backend, and the model is prepared for quantization using **torch.quantization.prepare()**. Subsequently, a new model (**quantized_model**) is formed as a sequence of quantization stubs, the original model, and dequantization stubs. The quantized model is then evaluated on the test dataset, and its accuracy is calculated and printed, providing insights into its performance on the given test data.

$$Q(x) = \text{Round}(\max(x) - \min(x) \times (\text{quant_range} - 1)) + \min_quant_value \quad (\text{vi})$$

$Q(x)$ is the quantized value of input x

$\min(x)$ is the minimum value in the input tensor x

$\max(x)$ is the maximum value in the input tensor x

quant_range is the range of quantization (i.e., for 8-bit quantization)

$\min_quant_value$ is the minimum quantized value.

References

- [1] X. Chen, Z.-L. Sun, & X. Chen, A machine learning based automatic tomato classification system. 2021 33rd Chinese Control and Decision Conference (CCDC) (2021).
- [2] H.-G. Choi, K.-S. Park, Ripening process of tomato fruits postharvest: impact of environmental conditions on quality and chlorophyll a fluorescence characteristics, *Horticulturae* 9 (7) (2023) 812.
- [3] A.N. de Oliveira, S.R.F. Bolognini, L.C. Navarro, J. Delafiori, G.M. Sales, D.N. de Oliveira, R.R. Catharino, Tomato classification using mass spectrometry-machine learning technique: a food safety-enhancing platform, *Food Chem.* 398 (2023) 133870.
- [4] M. Dinu, R. Soare, C. Babeanu, G. Hoza, M. Botu, Effects of organic farming system on some nutritional parameters of tomatoes fruits at different ripening stages, *Chil. J. Agric. Res.* 83 (3) (2023) 293–306.
- [5] X. Du, Z. Meng, Z. Ma, W. Lu, H. Cheng, Tomato 3D pose detection algorithm based on keypoint detection and point cloud processing, *Comput. Electron. Agric.* 212 (2023) 108056.
- [6] M. Filip, T. Zoubek, R. Bumbalek, P. Cerny, C.E. Batista, P. Olsan, P. Bartos, P. Kriz, M. Xiao, A. Dolan, Advanced computational methods for agriculture machinery movement optimization with applications in sugarcane production, *Agriculture* 10 (10) (2020) 434.
- [7] M.B. Garcia, S. Ambat, & R.T. Adao, Tomayto, tomahto: A machine learning approach for tomato ripening stage identification using pixel-based color image classification. 2019 IEEE 11th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM) (2019).
- [8] K. He, X. Zhang, S. Ren, & J. Sun, Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016).
- [9] K. Jha, A. Doshi, P. Patel, M. Shah, A comprehensive review on automation in agriculture using artificial intelligence, *Artif. Intell. Agric.* 2 (2019) 1–12.
- [10] T. Khatun, A. Razzak, M.S. Islam, M.S. Uddin, An extensive real-world in field tomato image dataset involving maturity classification and recognition of fresh and defect tomatoes, *Data Brief.* 51 (2023) 109688.
- [11] Q. Lelasseux, *Assessing the Impact of Including Diseased Crop Images in Training Datasets on the Performance of Convolutional Neural Networks for Crop Classification* University of Twente] (2025).
- [12] P. Li, J. Zheng, P. Li, H. Long, M. Li, L. Gao, Tomato maturity detection and counting model based on MHSA-YOLOv8, *Sensors* 23 (15) (2023) 6701.
- [13] R. Li, Z. Ji, S. Hu, X. Huang, J. Yang, W. Li, Tomato maturity recognition model based on improved YOLOv5 in greenhouse, *Agronomy* 13 (2) (2023) 603.
- [14] Y. Li, C. Li, T. Zhu, S. Zhang, L. Liu, Z. Guan, A recognition model for winter peach fruits based on improved ResNet and multi-scale feature fusion, *Front. Plant Sci.* 16 (2025) 1545216.
- [15] E. Limonova, D. Alfonso, D. Nikolaev, V.V. Arlazarov, ResNet-like architecture with low hardware requirements, 2020 25th Int. Conf. Pattern Recognit. (ICPR) (2021).
- [16] Z. Liu, Z. Shen, S. Li, K. Helwegen, D. Huang, K.-T. Cheng, How do adam and training strategies help bnns optimization, *Int. Conf. Mach. Learn.* (2021).
- [17] N. Lumoring, A.Y. Zakkiyah, F.I. Kurniadi, K.A. Minor, Classification of tomato maturity using transfer learning efficientnetB0 algorithm. 2024 8th international conference on information technology, Inf. Syst. Electr. Eng. (ICITISEE) (2024).
- [18] Y. Majeed, M. Waseem, Postharvest handling systems, In *Encyclopedia of smart agriculture technologies*, Springer, 2022, pp. 1–9.
- [19] P. Molchanov, S. Tyree, T. Karras, T. Aila, J. Kautz, Pruning convolutional Neural Netw. Resour. Effic. Inference arXiv Prepr. arXiv 1611 (2016) 06440.
- [20] H.S. Mputu, A. Abdel-Mawgood, A. Shimada, M.S. Sayed, Tomato quality classification based on transfer learning feature extraction and machine learning algorithm classifiers, *IEEE Access* (2024).
- [21] D. Nath, J. Roy, Forecast of Movie Sentiment Based on Multi Label Text Classification on Rotten Tomatoes Using Multiple Machine and Deep Learning Technique. *International Conference on Computer, Communication, and Signal Processing*, 2023.
- [22] W.D.N. Pacheco, & F.R.J. López, Tomato classification according to organoleptic maturity (coloration) using machine learning algorithms K-NN, MLP, and K-Means Clustering. 2019 XXII symposium on image, signal processing and artificial vision (STSIVA) (2019).
- [23] J. Poyatos, D. Molina, A. Martínez-Seras, J. Del Ser, F. Herrera, Multiobjective evolutionary pruning of Deep Neural Networks with Transfer Learning for improving their performance and robustness, *Appl. Soft Comput.* 147 (2023) 110757.
- [24] R. Qasrawi, M. Amro, R. Zaghal, M. Sawafteh, S.V. Polo, Machine learning techniques for tomato plant diseases clustering, prediction and classification, 2021 Int. Conf. Promis. Electron. Technol. (ICPET) (2021).
- [25] L.-D. Quach, K.N. Quoc, A.N. Quynh, H.T. Ngoc, N.T. Nghe, Tomato health monitoring system: tomato classification, detection, and counting system based on YOLOv8 model with explainable mobilenet models using grad-CAM, *IEEE Access* (2024).
- [26] R. Rahman, Using Machine Learning to Identify Crop Diseases with ResNet-18.
- [27] O.-R. Rusu, I. Mangalagiu, D. Amăriucăi-Mantu, G.-C. Teliban, A. Cojocaru, M. Burducea, G. Mihalache, M. Roșca, G. Caruso, A. Sekara, Interaction effects of cultivars and nutrition on quality and yield of tomato, *Horticulturae* 9 (5) (2023) 541.
- [28] R.E. Serge Ndayitabi, *Tomato: maturity indices and quality*. D. c. The Regents of the University of California. (2020). <<https://horticulture.ucdavis.edu/information/tomato-maturity-indices-and-quality>>.
- [29] S. Shezi, L. Magwaza, K. Ncama, Importance of maturity indexing in postharvest management of selected subtropical and temperate fruits, *Postharvest Management of Fresh Produce*, Elsevier, 2023, pp. 59–79.
- [30] R. Shinoda, H. Kataoka, K. Hara, R. Noguchi, Transformer-based ripeness segmentation for tomatoes. *Smart Agricultural, Technology* 4 (2023) 100196.

- [31] F. Su, Y. Zhao, G. Wang, P. Liu, Y. Yan, L. Zu, Tomato maturity classification based on SE-YOLOv3-MobileNetV1 network under nature greenhouse environment, *Agronomy* 12 (7) (2022) 1638.
- [32] USDA. *United States Standards for Grades of Fresh Tomatoes*. (1991). <<https://www.ams.usda.gov/grades-standards/tomato-grades-and-standards>>.
- [33] H.-T. Vo, K.C. Mui, N.N. Thien, P.P. Tien, Automating tomato ripeness classification and counting with YOLOv9, *Int. J. Adv. Comput. Sci. Appl.* 15 (4) (2024).
- [34] M. Wakchaure, B. Patle, A. Mahindrakar, Application of AI techniques and robotics in agriculture: a review, *Artif. Intell. Life Sci.* (2023) 100057.
- [35] M. Wu, H. Lin, X. Shi, S. Zhu, B. Zheng, MTS-YOLO: a multi-task lightweight and efficient model for tomato fruit bunch maturity and stem detection, *Horticulturae* 10 (9) (2024) 1006.
- [36] J. Zhang, J. Xie, F. Zhang, J. Gao, C. Yang, C. Song, W. Rao, Y. Zhang, Greenhouse tomato detection and pose classification algorithm based on improved YOLOv5, *Comput. Electron. Agric.* 216 (2024) 108519.